Quentin Scheetz

Term Project: Instant Messaging App

Test Plan Document

Table of Contents

1 Introduction

This test plan document outlines the comprehensive testing strategy, scope, schedule, and release criteria for the Instant Messaging App term project. It details the unit, feature, and system testing phases required to ensure the reliable performance of the backend server, real-time Socket.IO communication, and the React user interface. The primary objective of this plan is to verify that all core functionalities, such as secure user authentication, channel role management, and real-time message delivery, function correctly and securely before final deployment.

1.1 Purpose

This document outlines the test plan for the student instant messaging app term project. The application consists of a chat server and multiple chat clients that allow users to send and receive real-time messages over the Internet.

1.2 Target Audience

The primary target audience for this document includes Prof. Fei Xie and TA Huan Wu.

1.3 Terms and Definitions

Channel: A group messaging space where multiple users can communicate with each other.
Chat: A one-on-one conversation between two users.
MERN: A technology stack consisting of MongoDB, Express, React, and Node.js.
Ngrok: A tool used to deploy the remote backend service and expose it to the Internet.
JWT: JSON Web Tokens, used for secure user authentication and authorization.
Socket.IO: A library utilized for real-time WebSocket communication and direct messaging.

2 Test Plan Description

This section outlines the testing strategy to validate the frontend user interface, the backend application services, and the shared infrastructure of the application.

2.1 Scope of Testing

Testing will cover the React frontend, Node.js/Express backend APIs, MongoDB database interactions, and Socket.IO real-time communications. Core functionalities to be tested include user accounts, adding users, channel communication, direct messaging, and blocking.

2.2 Testing Schedule

Phase	Testing Activity	Focus Areas
Phase 1	**Unit Testing**	Backend APIs (/api/auth, /api/messages), MongoDB Mongoose schemas, password hashing, JWT generation, and Socket.IO internal mapping.
Phase 2	**Integration & Feature Testing**	React frontend integration with Express backend. Testing Epic stories.

Phase 3	**System & Security Testing**	End-to-end environment testing. Verifying Ngrok/CORS, checking JWT token security, and vulnerability checks against injection attacks.
Phase 4	**Bug Fixing & Final Review**	Resolving issues discovered in earlier phases, performing usability tests, and verifying that all minimum release criteria are met for Prof. Fei Xie and TA Huan Wu.

2.3 Release Criteria

The system must be fully capable of handling cross-origin requests without CORS errors or Ngrok security splash page blocks. User data must be securely protected from unauthorized access using JSON Web Tokens. All epic user stories must meet their acceptance criteria, such as messages appearing instantly for all participants without a page refresh.

3 Unit Testing

Unit testing will verify the individual logical divisions of the backend API routes and database schemas. In order to run these tests, we will utilize the Jest framework.

3.1 Auth Route (/api/auth)

These tests handle the backend logic for user registration, login, and secure access to endpoints using JSON Web Tokens (JWT). The scope of testing includes Express routing, bcrypt password hashing, and token generation. The minimum criteria for release are that valid credentials successfully generate an auth token stored in a cookie, and invalid credentials are rejected safely.

3.1.1 Valid Signup

Input: An email and a password.

Expected Output: The password is hashed using bcrypt, a JWT token is generated, and the token is stored in a cookie.

3.1.2 Protected Endpoint Access

Input: An API request containing a valid JWT cookie.

Expected Output: The verifyToken middleware successfully authenticates the request and grants access to the endpoint.

3.2 Messages Route (/api/messages)

This unit handles CRUD operations on chat history stored in MongoDB Atlas. The scope of testing covers the Mongoose schema validation and database insertion/retrieval functions. The minimum criteria for release are that messages must contain a sender, recipient, text content, messageType, and timestamp without triggering database errors.

3.2.1 Message Schema Validation

Input: A message containing sender, recipient, text content, messageType, and a timestamp.

Expected Output: Data is successfully stored and retrieved from MongoDB Atlas to maintain an accurate conversation history.

3.3 Socket.IO Mapping

This unit handles the WebSocket handshakes and the mapping of active connections. The scope of testing includes the server-side Socket.IO instance and its ability to store the active userId to socket.id pairs. The minimum criteria for release is that incoming connections correctly register their user ID so that private messages can be routed.

3.3.1 User Socket Connection

Input: An incoming user's userId.

Expected Output: The backend maps the ID to their socket connection to ensure messages are routed to the correct sender and recipient.

4 Feature Testing

Feature testing verifies that the application correctly implements the epic user stories designed to address student pain points, ensuring a smooth, distraction-free environment for studying and collaboration. We used Jest for automated backend testing and manual browser testing of the React frontend components. Overall, we are highly satisfied with the outcome, as the core messaging functionalities perform with low latency.

4.1 User Registration & Authentication

This feature allows students to create accounts, log in, and securely access the platform. The functionality includes a landing page UI with forms for username and password. The scope of testing covers the frontend UI form validation, the Axios POST request to the backend, and the handling of the returned JWT token. The minimum criteria for release are that a student can successfully create an account and is redirected to the main application dashboard upon success. We performed an interaction test to ensure the UI correctly communicates with the backend APIs.

4.1.1 Account Creation

Input: A student provides a username and password on the landing page and hits "Sign Up".

Expected Output: The account is created, allowing the user to add friends and join channels.

4.2 Real-time Channel Messaging

This feature serves as the core communication method, allowing multiple users to exchange messages within a shared space instantly. The scope of testing involves the React chat interface, Socket.IO message emission, and the broadcasting of that message to all clients in the channel. The minimum criteria for release is that messages must appear for all participants without requiring a page refresh, overcoming any Ngrok/CORS blocking. We performed an interaction test to verify real-time data flow.

4.2.1 Instant Message Delivery

Input: A user types a message inside a channel or direct chat and hits enter.

Expected Output: The message appears instantly for all participants without a page refresh.

4.3 Channel Creation & Role Management

This feature provides an interface for creating public/private channels and assigning an "Owner" status to the creator. The scope of testing includes the channel creation modal, API requests to create the channel entity in MongoDB, and the UI rendering of owner-specific controls (like delete or kick). The minimum criteria for release are that the creator is strictly given owner privileges, and standard members cannot access these destructive actions. We performed a usability test to ensure this feature successfully allows students to manage their own group project spaces effectively.

4.3.1 Owner Privileges

Input: A logged-in user clicks "Create Channel" and invites specific contacts.

Expected Output: A new messaging space is formed, and the creator is assigned "Owner" status. Only the owner can delete the channel or remove members.

4.3.2 Editing/Deleting Messages

Input: A user clicks on their own sent message to modify the text or remove it entirely.

Expected Output: The message updates or deletes successfully, but users cannot edit or delete messages sent by others.

5 System Testing

System testing will validate the application as a whole, focusing on the integration of the React frontend, Express APIs, Socket.IO, and MongoDB.

- **Security Testing:** Ensure the system is safe from injection attacks, cross-site scripting attacks, session hijacking attacks, and brute force attacks. This heavily relies on validating that JWT tokens securely prevent unauthorized access.
- **Configuration & Deployment Testing:** A key test is ensuring the system bypasses Ngrok restrictions. We must verify that the frontend explicitly includes the "ngrok-skip-browser-warning": "true" header and withCredentials: true. Without these, cross-origin cookies, tokens, and WebSocket handshakes will fail.
- **End-to-End Environment Testing:** Confirm that the application runs smoothly in standard web browsers (Chrome, Firefox, Safari, Edge) with JavaScript enabled.